

Session 14

Git Branching, Merging & Practical Workflow

What are you going to learn in this session?

In this session, you will move from basic Git usage to real-world workflows by learning how branching and merging work. You will understand how to create separate lines of work, make changes safely, and then combine them back into your main project.

You will also learn how Git stores changes internally as snapshots, how to view differences, and how to recover from common mistakes using simple undo and stash commands. By the end of the session, you will complete a guided workflow that mirrors how teams actually use Git.

Why is this topic important?

In real projects, you rarely work directly on the main codebase. Developers create branches to build features, fix bugs, and experiment without breaking the stable version. Understanding branching and merging is essential for collaboration and safe development.

This session bridges the gap between basic commands and real workflows. It helps you think like a developer who manages changes, isolates work, and integrates updates confidently.

How should you prepare yourself to attend this session?

Before attending this session, ensure you are comfortable with `git init`, `git add`, `git commit`, and `git log`. Your project folder should already be a Git repository with at least one commit.

Keep your terminal open inside your project directory and be ready to type along. This session is highly hands-on, and you will learn best by executing each step and observing results.

What should you do after the session?

After the session, you should practice creating branches for small changes and merging them back into the main branch. Try creating simple scenarios where you make changes in different branches and then combine them.

You should also practice resolving conflicts and using `git diff` to inspect changes before committing. Over time, this will become your default workflow while building projects.

Class Notes

How Git Works Internally (Simple Mental Model)

Git does not store changes as differences between files in the way many beginners imagine. Instead, Git stores snapshots of your project at each commit, which represent the complete state of your files at that moment.

Each commit is linked to the previous one, forming a chain of history. When you make a new commit, Git records a new snapshot and connects it to the previous snapshot, allowing you to move backward and forward in time.

Shell

```
git add .  
git commit -m "save current state"
```

This mental model helps you understand why commits are powerful and why frequent commits make your work easier to manage.

Creating Repositories Properly

A clean workflow starts with creating a project folder first and then initializing Git inside it. This ensures your repository structure is organized from the beginning.

Shell

```
mkdir my-project  
cd my-project  
git init
```

It is good practice to make your first commit early so that Git starts tracking changes immediately. Frequent commits help you avoid losing work and make debugging easier later.

Shell

```
git add .  
git commit -m "initial commit"
```

Branching Concept (Core Idea)

A branch is a separate line of development where you can work independently of the main version. The default branch is usually called `main`, which represents the stable version of your project.

You can think of branches as safe spaces for experimentation. Instead of modifying the main code directly, you create a branch, make changes there, and only merge them back once everything works correctly.

This approach prevents breaking the main version and allows multiple developers to work in parallel.

Working with Branches (Hands-On)

You can create a new branch and switch to it using the following commands. Once you are on the new branch, any changes you make will remain isolated from the main branch.

Shell

```
git branch feature-1  
git switch feature-1
```

Alternatively, you can create and switch in one step:

Shell

```
git switch -c feature-1
```

After switching, make a small change in your project and commit it. You will notice that this change exists only in the branch and not in the main version.

Merging Branches

Once your work in a branch is complete, you can merge it back into the main branch. This combines the changes from the branch into the main project.

Shell

```
git switch main  
git merge feature-1
```

After merging, the changes made in `feature-1` become part of `main`. You can verify this using `git log` to see the updated history.

Merge Conflicts (Important)

Merge conflicts occur when the same part of a file is changed differently in two branches. Git cannot decide which version to keep, so it asks you to resolve the conflict manually.

When a conflict occurs, you will see markers inside your file indicating the conflicting changes.

```
None
<<<<<< HEAD
current change
=====
branch change
>>>>>> feature-1
```

You need to edit the file, remove the markers, and keep the correct version. After resolving, you stage and commit the changes.

```
Shell
git add .
git commit -m "resolve conflict"
```

Conflicts are normal and part of development, so the goal is to understand how to resolve them calmly.

Viewing Changes with `git diff`

The `git diff` command helps you see what has changed in your files before committing. This is useful for reviewing your work and avoiding mistakes.

```
Shell
git diff
```

You can also check staged changes using:

```
Shell
git diff --staged
```

This helps you understand exactly what will be included in your next commit.

Undo Basics with git reset

Sometimes you may stage files by mistake or want to undo changes before committing. The `git reset` command allows you to move changes out of the staging area.

```
Shell
git reset file.txt
```

This does not delete your changes, it simply removes them from the staging area so you can review or modify them again.

Temporary Save with git stash

There are situations where you are working on something incomplete but need to switch branches. Instead of committing unfinished work, you can temporarily store it using stash.

```
Shell
git stash
git stash pop
```

Stash saves your current changes and cleans your working directory. Later, you can restore those changes using `git stash pop`.

Practical Branching Workflow Exercise

You will now perform a structured workflow that mirrors real development practices. Start by creating a new branch for a feature, make changes, commit them, and then merge back into the main branch.

```
Shell
git switch -c feature-ui
# make changes in HTML/CSS
git add .
git commit -m "update UI"

git switch main
git merge feature-ui
```

Optionally, you can create a conflict scenario and resolve it to reinforce your understanding. Completing this flow once will make Git workflows much clearer.

Common Beginner Confusions

Many learners confuse commits and branches, but they serve different purposes. Commits save snapshots of changes, while branches create separate paths of development.

Merge conflicts often feel intimidating, but they are simply situations where Git needs your help to choose the correct version. Understanding the markers makes resolution straightforward.

Commands like `reset` and `stash` may seem risky, but when used correctly, they are very helpful tools for managing work in progress and avoiding mistakes.

Key Takeaways

You now understand how Git stores project history as snapshots and how branching allows you to work safely on new features. You have learned how to create branches, switch between them, merge changes, and handle conflicts.

You have also explored tools like `git diff`, `git reset`, and `git stash`, which help you manage changes more effectively. These workflows are used in real-world development, and practicing them regularly will build your confidence quickly.

At this stage, your focus should be repetition and hands-on practice. The more you use Git in your daily workflow, the more natural these concepts will become.